

M10 - Docker + Airflow

19-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Package work with Docker and orchestrate reliable workflows with Airflow concepts and evidence.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M05 Python and M06 Linux/Git foundations. You should be able to run a command, inspect logs/files, and work from a reproducible project folder.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE10-01 - Why containers exist	5
DE10-02 - Images and containers	7
DE10-03 - Writing a Dockerfile	9
DE10-04 - Ports	11
DE10-05 - Volumes and persistence	13
DE10-06 - Networks and service communication	15
DE10-07 - Compose, configuration and secrets	17
DE10-08 - Why orchestration exists	19
DE10-09 - Airflow architecture and UI	21
DE10-10 - DAGs, tasks and dependencies	23
DE10-11 - TaskFlow, operators and task communication	25
DE10-12 - Scheduling and data intervals	27
DE10-13 - Retries, timeouts and failure behavior	29
DE10-14 - Parameters and task communication	31
DE10-15 - Connections, variables and secrets	33
DE10-16 - Sensors and waiting	35
DE10-17 - Logs and debugging	37
DE10-18 - Catchup, backfills and reprocessing	39
DE10-19 - Integration and production-style orchestration	41

Prerequisite, tool and outcome map

ASSUMED

Complete M05 Python and M06 Linux/Git foundations. You should be able to run a command, inspect logs/files, and work from a reproducible project folder.

MODULE PURPOSE

Package work with Docker and orchestrate reliable workflows with Airflow concepts and evidence.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE10-01 - Why containers exist

1. Why it matters

Explain what a container isolates and what it does not. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Explain what a container isolates and what it does not.

3. Prerequisite / what you already know

Complete M05 Python and M06 Linux/Git foundations. You should be able to run a command, inspect logs/files, and work from a reproducible project folder.

4. Picture it in your head

Picture “works on my machine” as an unpacked toolbox with unknown versions. A container image is the packing list; a running container is one unpacked instance created from that same list.

5. New words before we use them

An image is packaged filesystem/config metadata; a container is a running instance that still uses the host kernel. Containers improve reproducibility, not magical security or persistence.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Compare running worker.py on the host with the declared Docker environment. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can explain which parts of the runtime are isolated by a container and which still come from the host kernel, and you can show the same bundled worker behaving consistently inside the declared container environment.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: Record docker image/container identity, the worker exit code/output, and the declared dependency/runtime version. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If you call a container a VM, restate kernel sharing and lifecycle.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture “works on my machine” as an unpacked toolbox with unknown versions. 3. Run the earliest independent control: Record docker image/container identity, the worker exit code/output, and the declared dependency/runtime version. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Using the bundled worker, record the host Python/runtime context, then run the worker in the course container. Write three differences that the image controls and two things a container does not automatically guarantee.

13. Validate your result

Record docker image/container identity, the worker exit code/output, and the declared dependency/runtime version. Confirm the container run succeeds from a fresh container and that your explanation does not claim VM-level isolation.

14. Teach it back in your own words

Without notes, explain Why containers exist in your own words. Start with the purpose: Explain what a container isolates and what it does not. Then name one realistic failure mode and explain how you would prove the result using this check: Record docker image/container identity, the worker exit code/output, and the declared dependency/runtime version.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Why containers exist with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-02.

DE10-02 - Images and containers

1. Why it matters

Distinguish immutable image layers from mutable container runtime state. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Distinguish immutable image layers from mutable container runtime state.

3. Prerequisite / what you already know

Prerequisite: DE10-01 mastered with saved evidence.

4. Picture it in your head

An image is an immutable blueprint/layer stack; a container is a running process/filesystem view created from the image. Many containers can come from one image.

5. New words before we use them

Build once, run many. Container changes disappear when the container is replaced unless persisted externally.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Trace FROM/COPY/RUN into image, then CMD into a container process. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can point to an image as the reusable immutable build artifact and a container as one running/mutable instance created from it.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: Use docker image ls/inspect and docker ps/inspect. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If you rely on changes made interactively inside a container, rebuild the image instead.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: An image is an immutable blueprint/layer stack; a container is a running process/filesystem view created from the image. 3. Run the earliest independent control: Use `docker image ls/inspect` and `docker ps/inspect`. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build the supplied image, start a container, create a temporary file inside the running container, then start a second fresh container from the same image and compare the runtime state.

13. Validate your result

Use `docker image ls/inspect` and `docker ps/inspect`. Prove the temporary runtime change belongs to the first container and did not rewrite the source image; save the image ID plus both container IDs.

14. Teach it back in your own words

Without notes, explain Images and containers in your own words. Start with the purpose: Distinguish immutable image layers from mutable container runtime state. Then name one realistic failure mode and explain how you would prove the result using this check: Use `docker image ls/inspect` and `docker ps/inspect`.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Images and containers with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-03.

DE10-03 - Writing a Dockerfile

1. Why it matters

Write a small, deterministic Dockerfile with a clear base, workdir, copy/install order and non-root runtime. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Write a small, deterministic Dockerfile with a clear base, workdir, copy/install order and non-root runtime.

3. Prerequisite / what you already know

Prerequisite: DE10-02 mastered with saved evidence.

4. Picture it in your head

A Dockerfile is a recipe read top-to-bottom to build an image layer stack. Each instruction changes the build context or runtime defaults deliberately.

5. New words before we use them

Cache-friendly build order copies dependency metadata before changing app code. Pin deliberately when reproducibility matters. Avoid secrets in build context/layers.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Inspect the provided Python 3.13-slim Dockerfile and .dockerignore. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

A small Dockerfile builds successfully, copies only the intended project files, installs declared dependencies, uses the intended work directory/user, and starts the expected command.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: Build with --no-cache once, save the final image ID/size, run the image from a fresh container, and confirm the expected worker output and exit code. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If builds leak local files, inspect build context and .dockerignore.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A Dockerfile is a recipe read top-to-bottom to build an image layer stack. 3. Run the earliest independent control: Build with --no-cache once, save the final image ID/size, run the image from a fresh container, and confirm the expected worker output and exit code. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

From a blank Dockerfile, containerize the bundled tiny worker using a slim Python base, explicit WORKDIR, dependency install, COPY order and a non-root runtime user. Build it with a new tag and run it.

13. Validate your result

Build with --no-cache once, save the final image ID/size, run the image from a fresh container, and confirm the expected worker output and exit code. Inspect the Dockerfile to prove no secret or unnecessary dataset was copied.

14. Teach it back in your own words

Without notes, explain Writing a Dockerfile in your own words. Start with the purpose: Write a small, deterministic Dockerfile with a clear base, workdir, copy/install order and non-root runtime. Then name one realistic failure mode and explain how you would prove the result using this check: Build with --no-cache once, save the final image ID/size, run the image from a fresh container, and confirm the expected worker output and exit code.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Writing a Dockerfile with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-04.

DE10-04 - Ports

1. Why it matters

Explain container ports, EXPOSE, host publishing and loopback binding. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Explain container ports, EXPOSE, host publishing and loopback binding.

3. Prerequisite / what you already know

Prerequisite: DE10-03 mastered with saved evidence.

4. Picture it in your head

A container port is a door inside the isolated container; publishing maps a host door to that container door so traffic can reach the service.

5. New words before we use them

EXPOSE documents intended container ports; it does not publish them. Host publication maps host:container and binding to 127.0.0.1 limits local exposure.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Read 127.0.0.1:8080:8080 in compose.yaml. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can distinguish a service listening inside the container from a host-published port and can reach the service only through the mapping you intentionally configured.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: Save docker ps port mapping and one successful HTTP response from the host. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If localhost cannot connect, separate app listen address, container port, published host port and firewall.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A container port is a door inside the isolated container; publishing maps a host door to that container door so traffic can reach the service. 3. Run the earliest independent control: Save docker ps port mapping and one successful HTTP response from the host. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Run the bundled HTTP demo on its container port, publish it to a different host port, then test the mapped host endpoint. Change the host port and repeat without changing the application port.

13. Validate your result

Save docker ps port mapping and one successful HTTP response from the host. Confirm the unmapped/old host port fails after the changed run and explain why EXPOSE alone does not publish a port.

14. Teach it back in your own words

Without notes, explain Ports in your own words. Start with the purpose: Explain container ports, EXPOSE, host publishing and loopback binding. Then name one realistic failure mode and explain how you would prove the result using this check: Save docker ps port mapping and one successful HTTP response from the host.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Ports with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-05.

DE10-05 - Volumes and persistence

1. Why it matters

Choose a named volume when data must outlive container replacement. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Choose a named volume when data must outlive container replacement.

3. Prerequisite / what you already know

Prerequisite: DE10-04 mastered with saved evidence.

4. Picture it in your head

Container writable layers are disposable. A volume is an external storage drawer mounted into the container so important state can outlive one container instance.

5. New words before we use them

Container writable layers are ephemeral. Volumes are engine-managed persistent stores; bind mounts expose host paths and fit different use cases.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Trace pgdata to `/var/lib/postgresql/data`. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Data written to the named volume survives container replacement, while data written only to the disposable container layer does not.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: Record the volume name, remove/recreate the container, and prove the mounted marker persists while the unmounted temporary file does not. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If data vanishes after recreation, prove where it was written.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Container writable layers are disposable. 3. Run the earliest independent control: Record the volume name, remove/recreate the container, and prove the mounted marker persists while the unmounted temporary file does not. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a named volume, write one marker through a container, remove that container, start a fresh container with the same volume and read the marker. Also create one unmounted temporary file for contrast.

13. Validate your result

Record the volume name, remove/recreate the container, and prove the mounted marker persists while the unmounted temporary file does not. Do not count a container that was merely restarted as replacement proof.

14. Teach it back in your own words

Without notes, explain Volumes and persistence in your own words. Start with the purpose: Choose a named volume when data must outlive container replacement. Then name one realistic failure mode and explain how you would prove the result using this check: Record the volume name, remove/recreate the container, and prove the mounted marker persists while the unmounted temporary file does not.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Volumes and persistence with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-06.

DE10-06 - Networks and service communication

1. Why it matters

Use service-name DNS for container-to-container communication. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Use service-name DNS for container-to-container communication.

3. Prerequisite / what you already know

Prerequisite: DE10-05 mastered with saved evidence.

4. Picture it in your head

A Docker network is a private switch connecting services. Containers should call each other by service/container DNS names and internal ports, not by guessing host addresses.

5. New words before we use them

Compose creates a project network by default; services on it resolve each other by service name. Do not hardcode changing container IPs.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

The api uses DB_HOST=db and both join backend. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Two services on the same Docker network communicate through service-name DNS and internal ports without hard-coding a host-machine address.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: Save network inspect membership plus one successful service-name connection. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If service A cannot reach B, inspect network membership, service name and listening port before host port mappings.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A Docker network is a private switch connecting services. 3. Run the earliest independent control: Save network inspect membership plus one successful service-name connection. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Use the bundled app/db pair on one user-defined network. Resolve the database by service name from the app container, then deliberately try localhost from the app and explain the different target.

13. Validate your result

Save network inspect membership plus one successful service-name connection. Show that localhost inside the app container refers to the app container itself, not the database service.

14. Teach it back in your own words

Without notes, explain Networks and service communication in your own words. Start with the purpose: Use service-name DNS for container-to-container communication. Then name one realistic failure mode and explain how you would prove the result using this check: Save network inspect membership plus one successful service-name connection.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Networks and service communication with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-07.

DE10-07 - Compose, configuration and secrets

1. Why it matters

Define a multi-service stack while separating normal configuration from secrets. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Define a multi-service stack while separating normal configuration from secrets.

3. Prerequisite / what you already know

Prerequisite: DE10-06 mastered with saved evidence.

4. Picture it in your head

Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. The secret belongs in configuration, not source code/logs.

5. New words before we use them

Compose manages services, networks and volumes. Sensitive values should not be committed in source; use secrets or an appropriate secret backend.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Inspect POSTGRES_PASSWORD_FILE and per-service secret grant. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

A multi-service Compose file starts reproducibly with ordinary configuration separated from secret material, and each service receives only the configuration it needs.

9. Why it worked

The behavior is trustworthy because the environment boundary is declared explicitly and verified from observable runtime state instead of relying on hidden machine-specific setup. Your independent check is: Run docker compose config and inspect the running services. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If a secret appears in Dockerfile, Git or logs, rotate it and redesign injection.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. 3. Run the earliest independent control: Run docker compose config and inspect the running services. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a changed Compose stack with two services, one named volume and one secret/config value. Move the sensitive value out of the compose source into the supported secret/env mechanism used by the course.

13. Validate your result

Run docker compose config and inspect the running services. Confirm the stack starts, the expected volume/network exist, and the sensitive value is not committed in the Compose file or printed in normal logs.

14. Teach it back in your own words

Without notes, explain Compose, configuration and secrets in your own words. Start with the purpose: Define a multi-service stack while separating normal configuration from secrets. Then name one realistic failure mode and explain how you would prove the result using this check: Run docker compose config and inspect the running services.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Compose, configuration and secrets with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-08.

DE10-08 - Why orchestration exists

1. Why it matters

Explain why orchestration exists above scripts/cron. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Explain why orchestration exists above scripts/cron.

3. Prerequisite / what you already know

Prerequisite: DE10-07 mastered with saved evidence.

4. Picture it in your head

Orchestration is a traffic controller for recurring dependent jobs: it knows what should run, in what order, when to retry, and what state each run reached.

5. New words before we use them

Orchestration models dependencies, schedules, state, retries, observability and reruns. It coordinates work; it does not make unsafe tasks idempotent.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Map M09 pipeline steps into wait -> extract -> validate -> transform -> load -> reconcile. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can turn a multi-step batch script into an orchestration graph with explicit dependencies, retry boundaries and observable task states instead of one opaque cron command.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: Check that every node has one clear responsibility, all dependencies are acyclic, the validation/reconciliation gates can block publication, and rerunning one task would not require rerunning unrelated successful work. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If retrying the DAG duplicates data, repair task/data semantics before increasing retries.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Orchestration is a traffic controller for recurring dependent jobs: it knows what should run, in what order, when to retry, and what state each run reached. 3. Run the earliest independent control: Check that every node has one clear responsibility, all dependencies are acyclic, the validation/reconciliation gates can block publication, and rerunning one task would not require rerunning unrelated successful work. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Take the M09 batch flow and draw a DAG for wait/input -> extract -> validate -> transform -> load -> reconcile. Mark which failures should stop downstream work and which tasks are safe to retry.

13. Validate your result

Check that every node has one clear responsibility, all dependencies are acyclic, the validation/reconciliation gates can block publication, and rerunning one task would not require rerunning unrelated successful work.

14. Teach it back in your own words

Without notes, explain Why orchestration exists in your own words. Start with the purpose: Explain why orchestration exists above scripts/cron. Then name one realistic failure mode and explain how you would prove the result using this check: Check that every node has one clear responsibility, all dependencies are acyclic, the validation/reconciliation gates can block publication, and rerunning one task would not require rerunning unrelated successful work.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Why orchestration exists with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-09.

DE10-09 - Airflow architecture and UI

1. Why it matters

Identify scheduler, DAG processing, executor/workers, metadata/API/UI roles at a practical level. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Identify scheduler, DAG processing, executor/workers, metadata/API/UI roles at a practical level.

3. Prerequisite / what you already know

Prerequisite: DE10-08 mastered with saved evidence.

4. Picture it in your head

Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence.

5. New words before we use them

The scheduler decides runnable work; executors/workers run tasks; metadata tracks state; UI/API expose control/inspection. Airflow 3 uses a stable Task SDK/public interface.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use the current UI reference to locate DAG runs, task logs and task states. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can identify the practical roles of scheduler, DAG processing/executor or workers, metadata database and UI, and trace one DAG run down to task-instance logs/state.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: Save the DAG/run/task identifiers and one task log excerpt or simulator state. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If UI says failed, read the specific task/run/log rather than restarting everything.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Airflow separates definition from execution: DAG code describes workflow structure, scheduler decides runnable tasks, workers execute, metadata DB records state, UI exposes evidence. 3. Run the earliest independent control: Save the DAG/run/task identifiers and one task log excerpt or simulator state. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Open the current Airflow UI or the bundled static simulator/reference. For one sample DAG run, locate the run, one task instance, its state and its log, then map each observed action to the responsible Airflow component.

13. Validate your result

Save the DAG/run/task identifiers and one task log excerpt or simulator state. Your explanation must distinguish scheduler decisions from worker execution and metadata/UI observation.

14. Teach it back in your own words

Without notes, explain Airflow architecture and UI in your own words. Start with the purpose: Identify scheduler, DAG processing, executor/workers, metadata/API/UI roles at a practical level. Then name one realistic failure mode and explain how you would prove the result using this check: Save the DAG/run/task identifiers and one task log excerpt or simulator state.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Airflow architecture and UI with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-10.

DE10-10 - DAGs, tasks and dependencies

1. Why it matters

Define a DAG as tasks plus dependency rules, not as a giant script. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Define a DAG as tasks plus dependency rules, not as a giant script.

3. Prerequisite / what you already know

Prerequisite: DE10-09 mastered with saved evidence.

4. Picture it in your head

A data test turns an expectation into executable evidence. Severity decides whether a failure blocks publication, warns, or routes repair; CI runs those checks before changes are promoted.

5. New words before we use them

Tasks should have clear inputs/outputs, bounded side effects and explicit dependencies. Independent tasks can run concurrently when safe.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Read m10_daily_orders.py dependency chain. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

The DAG contains small tasks connected by explicit dependency rules; the graph communicates execution order without burying the whole pipeline in one giant task.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: Parse/import the DAG without error, inspect the graph or task dependency list, and prove validate is upstream of transform/publish. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If ordering is only hidden in code, make dependency structure explicit.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A data test turns an expectation into executable evidence. 3. Run the earliest independent control: Parse/import the DAG without error, inspect the graph or task dependency list, and prove validate is upstream of transform/publish. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a four-task DAG for extract -> validate -> transform -> publish, with validation blocking downstream work. Add one branch-independent notification or reconciliation task only if its dependency rule is explicit.

13. Validate your result

Parse/import the DAG without error, inspect the graph or task dependency list, and prove validate is upstream of transform/publish. Count the tasks and confirm there is no cycle.

14. Teach it back in your own words

Without notes, explain DAGs, tasks and dependencies in your own words. Start with the purpose: Define a DAG as tasks plus dependency rules, not as a giant script. Then name one realistic failure mode and explain how you would prove the result using this check: Parse/import the DAG without error, inspect the graph or task dependency list, and prove validate is upstream of transform/publish.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for DAGs, tasks and dependencies with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-11.

DE10-11 - TaskFlow, operators and task communication

1. Why it matters

Use TaskFlow for Python task composition and keep task communication small. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Use TaskFlow for Python task composition and keep task communication small.

3. Prerequisite / what you already know

Prerequisite: DE10-10 mastered with saved evidence.

4. Picture it in your head

A variable is a labeled box pointing to a value; the data type describes what operations make sense on that value. Operators combine values according to those type rules.

5. New words before we use them

TaskFlow automatically wires returned values through Airflow task communication. XCom-style payloads are control/small metadata, not a replacement for data storage.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

OPEN VIDEO / SOURCE - CLICK HERE (youtube.com)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Extract returns rows, load returns a summary, reconcile consumes summary. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

TaskFlow or operators compose Python work cleanly while task communication carries only small metadata/results rather than large datasets.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: Inspect task dependencies and the actual communicated payload. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If huge data is passed between tasks, persist it externally and pass a reference.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A variable is a labeled box pointing to a value; the data type describes what operations make sense on that value. 3. Run the earliest independent control: Inspect task dependencies and the actual communicated payload. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build a changed TaskFlow example where extract returns a small batch summary, transform consumes that summary/reference, and reconcile receives only the metrics it needs. Keep bulk data in a file/table, not task metadata.

13. Validate your result

Inspect task dependencies and the actual communicated payload. Confirm the payload is small/serializable, the downstream task receives the expected values, and large row data remains in durable storage.

14. Teach it back in your own words

Without notes, explain TaskFlow, operators and task communication in your own words. Start with the purpose: Use TaskFlow for Python task composition and keep task communication small. Then name one realistic failure mode and explain how you would prove the result using this check: Inspect task dependencies and the actual communicated payload.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for TaskFlow, operators and task communication with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-12.

DE10-12 - Scheduling and data intervals

1. Why it matters

Reason about schedule, logical date/data interval, start date and manual runs. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Reason about schedule, logical date/data interval, start date and manual runs.

3. Prerequisite / what you already know

Prerequisite: DE10-11 mastered with saved evidence.

4. Picture it in your head

A schedule creates logical data intervals. Think “this run processes the interval it represents,” not simply “run happened at this clock time.”

5. New words before we use them

A scheduled run represents a data interval, not simply the wall-clock instant the task starts. Use UTC-aware reasoning and explicit interval-based data queries.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

For a daily DAG, state which interval a run should process. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can distinguish wall-clock trigger time from the logical data interval a scheduled run represents and can state exactly which date/time slice a daily run should process.

9. Why it worked

This works because reason about schedule, logical date/data interval, start date and manual runs is made explicit before the result is accepted. Your independent check is: Record the logical date/data interval for the changed run and compare it with the partition/date actually processed. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If a run reads now() blindly, backfills may process the wrong slice.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A schedule creates logical data intervals. 3. Run the earliest independent control: Record the logical date/data interval for the changed run and compare it with the partition/date actually processed. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

For a daily DAG, write the logical interval for three consecutive scheduled runs and one manual run. Change the start date or requested interval and predict which data partition each run should read.

13. Validate your result

Record the logical date/data interval for the changed run and compare it with the partition/date actually processed. A run is wrong if it executes successfully but reads the wrong interval.

14. Teach it back in your own words

Without notes, explain Scheduling and data intervals in your own words. Start with the purpose: Reason about schedule, logical date/data interval, start date and manual runs. Then name one realistic failure mode and explain how you would prove the result using this check: Record the logical date/data interval for the changed run and compare it with the partition/date actually processed.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Scheduling and data intervals with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-13.

DE10-13 - Retries, timeouts and failure behavior

1. Why it matters

Configure retries for transient failures and timeouts for bounded execution. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Configure retries for transient failures and timeouts for bounded execution.

3. Prerequisite / what you already know

Prerequisite: DE10-12 mastered with saved evidence.

4. Picture it in your head

Treat the API as a contract with bounded waiting: request shape, success/error status, response schema, and timeout behavior all belong to ingestion logic.

5. New words before we use them

Retry only plausibly transient work. A retry is safe only when side effects are idempotent/transactional. `execution_timeout` bounds a task attempt.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Simulator extract fails twice then succeeds on attempt 3; validation failure does not retry blindly. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Transient failures retry according to policy, long/hung work is bounded by timeout, and deterministic failures eventually surface as failed rather than looping forever.

9. Why it worked

This works because configure retries for transient failures and timeouts for bounded execution is made explicit before the result is accepted. Your independent check is: Save try numbers, timestamps/state transitions and final task state for both cases. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If retries amplify damage, stop and repair idempotency or classify permanent errors.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Treat the API as a contract with bounded waiting: request shape, success/error status, response schema, and timeout behavior all belong to ingestion logic. 3. Run the earliest independent control: Save try numbers, timestamps/state transitions and final task state for both cases. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Use the simulator or a tiny task that fails once then succeeds to demonstrate retry count/delay. Create a second changed task that exceeds a short timeout or always fails.

13. Validate your result

Save try numbers, timestamps/state transitions and final task state for both cases. Confirm the transient case succeeds only within the configured retry budget and the permanent/hung case stops at the declared boundary.

14. Teach it back in your own words

Without notes, explain Retries, timeouts and failure behavior in your own words. Start with the purpose: Configure retries for transient failures and timeouts for bounded execution. Then name one realistic failure mode and explain how you would prove the result using this check: Save try numbers, timestamps/state transitions and final task state for both cases.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Retries, timeouts and failure behavior with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-14.

DE10-14 - Parameters and task communication

1. Why it matters

Use runtime Params for validated run configuration and task communication for outputs. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Use runtime Params for validated run configuration and task communication for outputs.

3. Prerequisite / what you already know

Prerequisite: DE10-13 mastered with saved evidence.

4. Picture it in your head

Operators/TaskFlow tasks are executable nodes. Pass small metadata between tasks; keep large datasets in durable storage and pass references rather than stuffing data into orchestration state.

5. New words before we use them

Params are configuration supplied/defaulted for a run and JSON-Schema validated. Task outputs carry produced metadata/results. Do not confuse them.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

min_amount is a validated DAG Param; task results form the simulator xcom dictionary. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Runtime parameters configure a run deliberately and task outputs communicate only the small values downstream tasks actually need.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: Show the accepted parameter appears in task behavior, the invalid value is rejected/handled as designed, and the communicated task output is small, explicit and tied to the changed run. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If a parameter changes business scope, record it in run metadata.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Operators/TaskFlow tasks are executable nodes. 3. Run the earliest independent control: Show the accepted parameter appears in task behavior, the invalid value is rejected/handled as designed, and the communicated task output is small, explicit and tied to the changed run. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Add a validated run parameter such as region/date to a small DAG and pass a compact summary from one task to the next. Run a changed parameter value and one invalid value.

13. Validate your result

Show the accepted parameter appears in task behavior, the invalid value is rejected/handled as designed, and the communicated task output is small, explicit and tied to the changed run.

14. Teach it back in your own words

Without notes, explain Parameters and task communication in your own words. Start with the purpose: Use runtime Params for validated run configuration and task communication for outputs. Then name one realistic failure mode and explain how you would prove the result using this check: Show the accepted parameter appears in task behavior, the invalid value is rejected/handled as designed, and the communicated task output is small, explicit and tied to the changed run.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Parameters and task communication with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-15.

DE10-15 - Connections, variables and secrets

1. Why it matters

Keep connection credentials and sensitive Variables out of DAG source. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Keep connection credentials and sensitive Variables out of DAG source.

3. Prerequisite / what you already know

Prerequisite: DE10-14 mastered with saved evidence.

4. Picture it in your head

Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. The secret belongs in configuration, not source code/logs.

5. New words before we use them

Connections identify external systems; secrets can come from environment/metastore/external backends. Masking helps but is not permission control.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Explain why `conn_id` belongs in DAG code but password does not. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

DAG source contains no plaintext credential; connections/variables/secrets provide runtime configuration with a clear boundary between ordinary and sensitive values.

9. Why it worked

This works because keep connection credentials and sensitive Variables out of DAG source is made explicit before the result is accepted. Your independent check is: Search the repository/DAG for the sample secret and confirm it is absent, then prove the task can still obtain the runtime value through the configured mechanism. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If plaintext credentials are committed, treat it as exposure and rotate.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. 3. Run the earliest independent control: Search the repository/DAG for the sample secret and confirm it is absent, then prove the task can still obtain the runtime value through the configured mechanism. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Move one sample endpoint/credential pair out of DAG source into the course-approved connection/env/secret mechanism. Keep a non-secret run setting as ordinary configuration for contrast.

13. Validate your result

Search the repository/DAG for the sample secret and confirm it is absent, then prove the task can still obtain the runtime value through the configured mechanism. Save only redacted evidence.

14. Teach it back in your own words

Without notes, explain Connections, variables and secrets in your own words. Start with the purpose: Keep connection credentials and sensitive Variables out of DAG source. Then name one realistic failure mode and explain how you would prove the result using this check: Search the repository/DAG for the sample secret and confirm it is absent, then prove the task can still obtain the runtime value through the configured mechanism.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Connections, variables and secrets with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-16.

DE10-16 - Sensors and waiting

1. Why it matters

Wait for external conditions without wasting worker capacity indefinitely. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Wait for external conditions without wasting worker capacity indefinitely.

3. Prerequisite / what you already know

Prerequisite: DE10-15 mastered with saved evidence.

4. Picture it in your head

A sensor is a waiting checkpoint: do not start downstream work until an external condition is true. Deferrable/poke behavior controls how expensive that waiting is.

5. New words before we use them

Sensors poll/wait. Reschedule or deferrable patterns can release worker resources; every wait needs a timeout/failure policy.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Simulator wait_input has bounded polls and fails downstream safely. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

A wait condition is bounded and observable: downstream work starts only when the external condition becomes true, and timeout/failure policy prevents indefinite worker occupation.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: Save poll/wait attempts and final state for both runs. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If a sensor waits forever, add timeout and decide skip/fail/escalate behavior.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A sensor is a waiting checkpoint: do not start downstream work until an external condition is true. 3. Run the earliest independent control: Save poll/wait attempts and final state for both runs. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Run the bundled wait-input simulator or sensor pattern once with the required input absent, then make the input available before the timeout. Repeat with a changed short timeout where the input never appears.

13. Validate your result

Save poll/wait attempts and final state for both runs. Confirm downstream work starts only after the condition is true in the success case and the missing-input case exits with the declared timeout/failure behavior.

14. Teach it back in your own words

Without notes, explain Sensors and waiting in your own words. Start with the purpose: Wait for external conditions without wasting worker capacity indefinitely. Then name one realistic failure mode and explain how you would prove the result using this check: Save poll/wait attempts and final state for both runs.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Sensors and waiting with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-17.

DE10-17 - Logs and debugging

1. Why it matters

Debug from task state and logs using a repeatable ladder. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Debug from task state and logs using a repeatable ladder.

3. Prerequisite / what you already know

Prerequisite: DE10-16 mastered with saved evidence.

4. Picture it in your head

Git has three useful areas: working tree -> staging area -> commit history. status tells where changes are; add selects the snapshot; commit records it.

5. New words before we use them

Start with failing task -> attempt -> exception -> input/config -> upstream state -> environment. Logs should include run/task identifiers but never secrets.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Read simulator logs for transient retries and validation failure. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can move from failed task state to the exact log/exception and isolate the earliest actionable cause instead of rerunning the whole DAG blindly.

9. Why it worked

This works because debug from task state and logs using a repeatable ladder is made explicit before the result is accepted. Your independent check is: Save failed task/run ID, the first actionable log line, the single repair made and the successful changed rerun. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If you rerun without identifying cause, you may hide an intermittent defect.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Git has three useful areas: working tree -> staging area -> commit history. 3. Run the earliest independent control: Save failed task/run ID, the first actionable log line, the single repair made and the successful changed rerun. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Inject one controlled failure into a changed task, run it, then use state/log evidence to identify the failing task and root-cause line. Fix only that cause and rerun the affected path.

13. Validate your result

Save failed task/run ID, the first actionable log line, the single repair made and the successful changed rerun. Do not count a full reset/rebuild with no diagnosis as debugging evidence.

14. Teach it back in your own words

Without notes, explain Logs and debugging in your own words. Start with the purpose: Debug from task state and logs using a repeatable ladder. Then name one realistic failure mode and explain how you would prove the result using this check: Save failed task/run ID, the first actionable log line, the single repair made and the successful changed rerun.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Logs and debugging with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-18.

DE10-18 - Catchup, backfills and reprocessing

1. Why it matters

Backfill historical intervals without changing business meaning or duplicating outputs. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Backfill historical intervals without changing business meaning or duplicating outputs.

3. Prerequisite / what you already know

Prerequisite: DE10-17 mastered with saved evidence.

4. Picture it in your head

A backfill replays historical input through the same controlled path. Reconciliation compares source receipts with landed/processed totals so missing or duplicated periods are visible.

5. New words before we use them

Backfills recreate runs for past data intervals. Tasks must use interval inputs and idempotent writes. Catchup is a scheduling choice; backfill is an explicit operation.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Simulator backfills three named dates and produces independent summaries. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Historical intervals can be reprocessed deliberately without changing their business meaning or creating duplicate outputs.

9. Why it worked

This works because backfill historical intervals without changing business meaning or duplicating outputs is made explicit before the result is accepted. Your independent check is: Compare partition/date counts and key totals before/after the backfill. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If historical rerun uses current date/config accidentally, stop and make interval/config explicit.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A backfill replays historical input through the same controlled path. 3. Run the earliest independent control: Compare partition/date counts and key totals before/after the backfill. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Choose two historical daily intervals in the bundled pipeline. Reprocess them using the same interval-aware logic, then run one of those intervals a second time to test rerun safety.

13. Validate your result

Compare partition/date counts and key totals before/after the backfill. The second run of the same interval must not create duplicate business rows or advance unrelated intervals.

14. Teach it back in your own words

Without notes, explain Catchup, backfills and reprocessing in your own words. Start with the purpose: Backfill historical intervals without changing business meaning or duplicating outputs. Then name one realistic failure mode and explain how you would prove the result using this check: Compare partition/date counts and key totals before/after the backfill.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Catchup, backfills and reprocessing with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE10-19.

DE10-19 - Integration and production-style orchestration

1. Why it matters

Defend an end-to-end orchestration design including deployment, state, recovery and observability. This matters because containers and orchestration run unattended; wrong dependencies, intervals, retries or configuration can turn one local mistake into a recurring operational failure.

2. What you are learning

Defend an end-to-end orchestration design including deployment, state, recovery and observability.

3. Prerequisite / what you already know

Prerequisite: DE10-18 mastered with saved evidence.

4. Picture it in your head

A backfill replays historical input through the same controlled path. Reconciliation compares source receipts with landed/processed totals so missing or duplicated periods are visible.

5. New words before we use them

Production readiness combines reproducible runtime, clear dependencies, safe secrets, bounded retries/waits, idempotent data operations, reconciliation, logs and tested reprocessing.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=5SFVDr6F4vg)

URL: <https://www.youtube.com/watch?v=5SFVDr6F4vg>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Combine Docker static lab with Airflow DAG + simulator evidence; identify what still requires real runtime. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

You can defend an end-to-end production-style orchestration design covering packaging, dependencies, scheduling, configuration/secrets, retries, state, backfill and observability.

9. Why it worked

The workflow is trustworthy because dependency, run interval, task state and failure behavior are explicit; the run evidence proves the orchestrator followed the intended graph rather than merely starting a script. Your independent check is: From a clean project state, prove the DAG imports, dependency graph is correct, changed run reaches the expected final state, validation can block publication, rerun/backfill is safe, and logs/evidence identify the run w. The proof is runtime/orchestrator state from a fresh run, not configuration that merely looks correct.

10. Common beginner mistakes

If a diagram cannot explain failure/recovery and evidence, it is not an operational design.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A backfill replays historical input through the same controlled path. 3. Run the earliest independent control: From a clean project state, prove the DAG imports, dependency graph is correct, changed run reaches the expected final state, validation can block publication, rerun/backfill is safe, and logs/evidence identify the run without exposing secrets. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Build a changed mini orchestration package for ingest -> validate -> transform -> publish -> reconcile. Include the Docker/runtime boundary, DAG dependencies, one retry policy, one secret/config rule, one historical reprocessing route and evidence locations.

13. Validate your result

From a clean project state, prove the DAG imports, dependency graph is correct, changed run reaches the expected final state, validation can block publication, rerun/backfill is safe, and logs/evidence identify the run without exposing secrets.

14. Teach it back in your own words

Without notes, explain Integration and production-style orchestration in your own words. Start with the purpose: Defend an end-to-end orchestration design including deployment, state, recovery and observability. Then name one realistic failure mode and explain how you would prove the result using this check: From a clean project state, prove the DAG imports, dependency graph is correct, changed run reaches the expected final state, validation can block publication, rerun/backfill is safe, and logs/evidence identify the run w.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Integration and production-style orchestration with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.